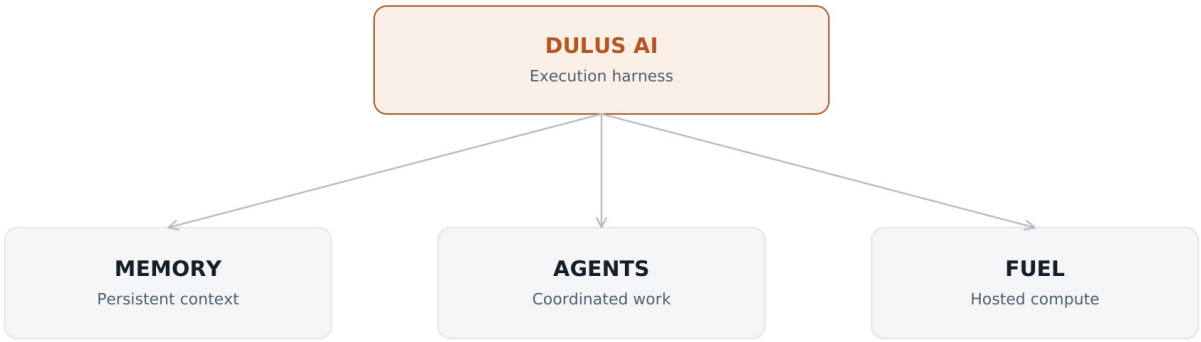
WHITEPAPER / EDITION 1.0

Dulus AI

Technical whitepaper and ecosystem overview

Agent harness and Fuel economics

An architecture for coordinating tools, models, and memory in real tasks, with a proposed evolution toward durable execution and an explanation of $DULUS utility in the hosted service.



Dulus AI is developed in Santo Domingo, Dominican Republic. Its name refers to Dulus dominicus, the palmchat. Kevin Rojo leads the project. [8]

October 19, 2026

Executive summary

Dulus AI is a harness that turns human goals into executable work through models, tools, memory, and specialized agents.

A model provides reasoning capability. The harness organizes the environment in which that capability becomes a completed task: it selects tools, preserves context, coordinates work, and presents verifiable results. Dulus places this execution layer at the center of the product and enables applications to be built on top of it.

The open-source foundation supports adoption and extension. The commercial proposition centers on services that reduce setup and operational effort, including hosted compute. $DULUS connects that service to the community through Fuel, the platform's consumption unit. Public documentation specifies a crediting ratio of 1 Fuel for every 1 $DULUS deposited. [2, 5]

The thesis of this edition

The ecosystem's value should develop alongside its ability to complete useful work. The proposed technical evolution applies durable execution to long-running tasks: preserving results, recovering pending operations, and avoiding repeated external effects. This architecture draws on Temporal's reference paper and adapts its principles to Dulus. [1, pp. 11 to 17]

The harness is the core of Dulus. Dulus Business is an application built on that core; Fuel is the consumption layer for hosted compute.

Content	Pages
Foundation and architecture	03 to 07
Applications and expansion	08
Token and Fuel economics	09 to 11
Evolution and validation	12 to 13
References and scope	14

Published capabilities and proposed designs are identified in each section. Future phases are a technical proposal, without calendar commitments.

The harness as a product

Dulus organizes execution around the task, independently of the provider supplying the model.

In a working session, the difficulty often appears after the first response. The agent must inspect files, query an application, execute an operation, interpret errors, and decide what to do next. Each transition needs information, state, and a way to determine whether the objective has been met.

Dulus is the infrastructure that keeps those transitions connected. A user can switch models without manually rebuilding the surrounding tool and memory environment. Its documentation supports external providers with the user's own keys, local runtimes, and inference hosted by Dulus. [3]

What this layer provides

Need	Harness response
Turn a request into actions	Break down the task and select tools with observable results.
Maintain continuity	Manage context and retrieve useful knowledge across sessions.
Handle specialized work	Delegate capabilities with defined inputs and outputs.
Correct an execution	Inspect results, apply changes, and verify the final outcome.

A common foundation for different applications

A developer can use Dulus for software maintenance; an operator, for business queries and processes; a community, for administrative automation. Each application defines its integrations and quality criteria around the same execution pattern.

The commercial opportunity is to reduce the effort required to put an agent to work and keep it operating. Evaluation should measure how much work it completes, what it costs, and how much intervention it needs. The number of available tools alone does not measure those outcomes.

Public product sources: [2, 3, 4]. Application examples describe possible uses of the harness, not adoption figures or customer contracts.

Reference architecture

The proposed architecture separates the user interface, work coordination, executable capabilities, and the state needed to continue.

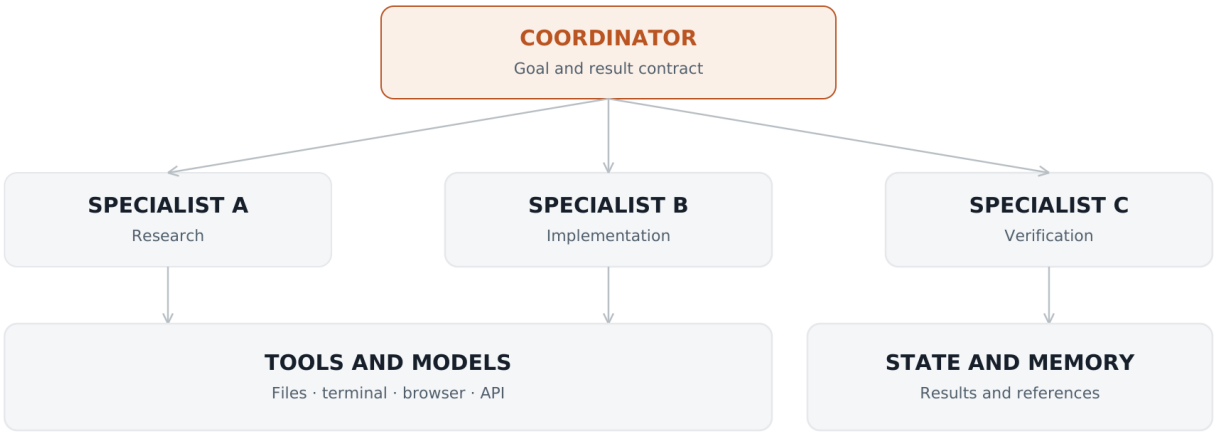


Figure 1 Proposed functional structure for an execution with specialists

The interface captures the objective and displays progress. The coordinator maintains the plan and determines which results are still needed. Capabilities execute work under an explicit contract; their tools and models stay close to the domain that uses them. Memory and state provide continuity throughout the process.

Layer	Responsibility
Interaction	Receive instructions and present progress and deliverables.
Coordination	Order steps, resolve dependencies, and assemble results.
Capabilities	Execute a bounded task with suitable tools and a suitable model.
State and memory	Preserve facts, results, references, and execution progress.
Compute and consumption	Connect providers and, for hosted inference, meter Fuel use.

An initial deployment may combine several layers in one process. Operational separation becomes useful when a capability needs to scale, recover, or evolve independently. The diagram expresses responsibilities; it does not require a separate service for every box.

Reference design adapted from the capability and execution boundaries described in [1, pp. 8 to 14 and 18 to 26]. It does not imply that a Temporal integration is already deployed in Dulus.

Multiagent coordination

A specialist is justified when it reduces a specific dependency of the coordinator and returns a result that can be checked.

Separating research, implementation, or verification can help when each type of work uses different sources, tools, or criteria. If every participant needs the entire conversation and the same instructions, the split adds communication without resolving pressure on context.

The proposed design starts with a capability contract. For example, a change review receives a repository reference, scope, and acceptance criteria. It returns findings with locations, evidence, and status. It also declares missing information and whether an error is retryable. [1, pp. 18 to 21]

Relationship	Proposed coordination model
One focused external action	An activity for a query, a model call, or a write.
A process with several steps	A subworkflow with its own state when it needs independent recovery and tracking.
A capability run by another operator	A versioned service with defined contracts, availability, and ownership.

Applying this to Dulus

Dulus already documents subagents and Mesa Redonda, its round-table collaboration mechanism. [4] In the proposed durable architecture, delegated work receives a stable identity, a structured result, and a completion state. The coordinator can distinguish a completed contribution from a partial response and act accordingly.

A round table is useful for exploring alternatives or reviewing a decision. Final acceptance should be checked against evidence: tests, files, sources, or the user’s criteria. Agreement between models does not replace that verification.

Known sequences belong in code. Model reasoning is used where the path depends on what is discovered during the task.

In a Temporal implementation, these relationships correspond to Activity, Child Workflow, and Nexus Operation. The choice depends on the capability’s lifecycle and ownership. [1, pp. 21 to 26]

Durable execution

A long-running task needs to preserve completed work and identify which operations remain pending after an interruption.

The proposal for Dulus separates flow control from calls that leave the process. The workflow retains the sequence and decisions; activities perform inference, queries, and external changes. Their recorded results allow progress to be reconstructed without requesting every completed response again. [1, pp. 12 to 16]

Execution event	Expected behavior
A provider rate-limits requests	Wait with increasing backoff while respecting the budget and deadline.
A worker process restarts	Recover persisted results and continue with pending work.
A decision arrives later	Record the event and resume the workflow without keeping a process blocked.
A write response is lost	Query the destination or reuse its idempotency key before repeating the operation.

Recovery example

Dulus prepares a software change: it inspects the project, applies a fix, and runs tests. If execution is interrupted while generating the report, recovery reuses recorded results. If the repository has changed before the run resumes, it first checks the revision and whether those results remain valid.

Durability does not make every operation exactly-once. An external system may accept a change before its confirmation is recorded. Every effect needs a stable identity and a deduplication strategy. The same caution applies to model calls: a recorded response can be reused; a billed call whose response was lost may require reconciliation.

Large documents remain outside the history through stable references. Validation errors are returned immediately; transient failures follow a bounded retry policy. This distinction prevents a fast failure from becoming repeated consumption.

Proposed evolution. Execution history, reconciliation, and recovery tests must be verified before continuity guarantees are announced.

Memory and context efficiency

Continuity depends on retrieving relevant information and keeping each decision’s required context small.

A conversation contains instructions, temporary results, extensive data, and decisions that remain useful later. Sending everything with every call increases cost and can crowd out important information. Dulus documents MemPalace as readable Markdown memory and Lookback and Loopback as mechanisms for accessing history. [2]

Three types of information

Type	Role in the architecture
Working context	The active goal, relevant instructions, and recent results.
Reusable memory	Facts and decisions with origin, scope, and dates where relevant.
Execution state	Completed steps, pending operations, and the identity of each external effect.

Memory and execution state serve different purposes. Remembering that a task was supposed to be done does not prove it was completed. To resume it, the coordinator needs the operation’s result, its version, and its relationship to the workflow. To work better in another session, it needs selected, updated knowledge.

Efficiency per completed task

Context improvements are evaluated on equivalent tasks: tokens sent, total cost, latency, and result quality. A reduction in tokens that forces work to be redone can worsen the experience. An additional query that prevents an incorrect edit can improve it.

In the proposed design, specialists receive references and bounded data. A summary links to the evidence needed to verify it. Extensive content remains in files or object storage and is retrieved when needed. This pattern keeps memory accessible without making it a permanent burden in every prompt. [1, pp. 14 and 19]

The efficiency metric is the cost of obtaining an acceptable result, including calls, retries, and corrective work.

Ecosystem applications

The harness supports different working environments while retaining a common model for tools, memory, and coordination.

Development and work on the computer

Dulus's public offering includes a desktop application, CLI, file and terminal tools, WebChat, and Telegram integration. [4] These interfaces let users start, observe, and direct a task from different access points. Consistent state is what must keep that experience coherent.

In software development, the result might be a fix accompanied by tests. In research, a report with sources and supporting files. In an administrative operation, a data update with verifiable confirmation. Each workflow needs its own completion criteria.

Dulus Business as an expansion path

Dulus Business is envisioned as an application of the harness for communities and small businesses. The project's product vision includes moderation, community-specific memory, support responses, customer management, and operational queries through messaging channels. Actual availability should be described by integration and version.

Proposed use case	Expected outcome
Community	Identify content that violates the rules and produce an operational summary.
Sales support	Retrieve customer information and prepare a contextual response.
Inventory and orders	Check stock, record authorized changes, and confirm status.
Business reporting	Assemble data and deliver a summary with the sources used.

Expansion priorities should follow the processes users repeat. A useful workflow template can become a maintained integration; a frequently used integration can justify a managed service. Commercial development can then build on observed needs and repeatable results.

The Dulus Business functions described here express the project's product vision. They are not a claim that all are available in production.

The DULUS token

$DULUS links participation in the ecosystem to a defined product utility: funding hosted inference consumption through Fuel.

Published identity	Detail
Name and symbol	Dulus AI · $DULUS
Network	Solana
Program and precision	Token-2022 · 6 decimals
Published official mint	9R8rrjXxcfQPmLTCLhmVpjr2uesjjkcgkinE6Lwdpump
Service unit	Fuel, the balance used to consume hosted services.

The identifiers above reflect information published by Dulus. [2, 5] The transferable token on Solana and the service balance are different concepts: depositing $DULUS credits Fuel to the user’s account. The crediting ratio does not mean that every agent call is a blockchain transaction.

Utility for users and developers

Fuel enables the use of hosted models through Dulus interfaces and its API. The developer integrates an inference service; the user maintains a balance to consume it. Utility is tied to service access and consumption.

Published holder access

The models page publishes a free tier for accounts with at least 1,000,000 $DULUS deposited as Fuel: 40 messages every 24 hours on dulus-free-9b, dulus-free-7b, and dulus-free-20b, with up to 2,048 output tokens per message. The required balance is not spent on these calls; tokens remaining in an external wallet do not qualify for this access. [3]

This access condition is not presented as yield-bearing staking. Holding the token does not, by itself, grant equity, profit-sharing, or governance rights. Any additional rights require express terms.

The utility described does not guarantee price appreciation or returns. This edition does not establish new allocations, emissions, lockups, or vesting schedules.

How Fuel works

Public documentation describes a cycle of deposit, reservation, execution, and settlement that connects the balance to model consumption. [5, 6]

A deposit is credited after confirmation on Solana. A balance is reserved before a hosted call; after completion, usage is settled and the unused portion is released. The published policy excludes cached tokens from billing and returns the reservation for failed calls. The exact charge is reported in X-Dulus-Fuel-Charged.

Stage	Effect on the balance
Deposit	Base credit of one Fuel for each $DULUS received.
Published bonus	An additional 1 percent credited as Fuel.
Reservation	A temporary hold before the call executes.
Settlement	Charge based on usage and the model rate; release of the remainder.

Illustrative accounting model

To represent consumption without fixing a commercial rate, let N be the total billable tokens, r the base rate in Fuel per token, and m the model multiplier. The following relationship summarizes the calculation structure; the current rate and the service receipt determine the actual charge.

$$F = N \times r \times m$$

Mathematical example: for N = 1,500 and m = 2, consumption would be 3,000 × r Fuel. Without r, there is no final price. Reasoning tokens must be counted without duplication if the provider already includes them in output.

An illustrative deposit of 100,000 $DULUS credits 100,000 base Fuel and, under the published 1 percent bonus, an additional 1,000 Fuel. This promotional credit is not an additional issuance of tokens on Solana or a promise of yield.

Fuel is a service accounting unit. The deposit ratio does not establish a fixed dollar price or a right to reverse conversion. Withdrawal or refund conditions must be checked in the applicable terms.

Ecosystem economics

Dulus’s economic thesis connects product adoption to a service need that users can measure.

The open-source edition lowers the friction of getting started and enables integrations. Hosted compute provides a path for users who prefer to delegate model operations. Business applications may expand that demand if they solve recurring processes. This describes a growth hypothesis, not a revenue forecast.

The relationship between product and token

Users who choose the hosted route need a balance to pay for that consumption. The token participates in funding Fuel. Utility alone does not determine market price: available supply, liquidity, circulation velocity, and participants’ decisions also matter.

Spending Fuel deducts a unit of service. It does not demonstrate that an onchain token is burned or permanently removed from circulation. This edition does not attribute buybacks, revenue sharing, usage-based emissions, or automatic supply destruction to the system.

Development driver	Indicator for validation
Useful harness activity	Completed tasks and repeat usage.
Preference for hosted service	Active users and Fuel consumption by cohort.
Experience quality	Retention, latency, and cost per accepted task.
Service economics	Net revenue relative to actual inference and operating costs.

Service sustainability

Infrastructure costs and a token-denominated balance may move differently. Operations must review rates, capacity, and bonuses to maintain a sustainable relationship between consumption and revenue. Higher usage with insufficient margins does not, by itself, constitute an economic improvement.

The growth proposal prioritizes reusable integrations, documentation that helps users get started, and evidence of results. The community contributes value through contributions, reports, and real use cases; a market quotation does not replace that validation.

Milestone based evolution

The proposed roadmap orders development around evidence of functionality and adoption, without assigning dates the team has not committed to.

Proposed phase	Deliverable and advancement criterion
1 Harness consolidation	Capability contracts, tool versions, and a representative task set. Advance when results can be reproduced and compared.
2 Durable continuity	An execution adapter with history, resumption, and effect reconciliation. Advance after demonstrating recovery from failures and no duplicate writes in tested scenarios.
3 Business applications	Bounded Dulus Business pilots with maintained integrations. Advance when the process completes useful work and operators choose to repeat it.
4 Service scale	Improvements to capacity, billing, and the consumption experience. Advance with retention, availability, and margin metrics sufficient to sustain growth.

Model research direction

One R&D direction contemplated by the project is adapting an open model to use tools within Dulus. Its usefulness would be evaluated in tool selection, result interpretation, error correction, and task completion. Model size would be selected according to quality, latency, and available resources.

Training or specialized fine-tuning would require authorized and reviewed trajectories, separation between training and evaluation data, and comparison with base models. A compact model would only be an improvement if it retained the capability needed to complete the work at lower cost or latency.

The engineering priority is to demonstrate continuity and quality on real tasks. A new integration, model, or utility enters the product when its contribution can be measured.

These phases describe a proposed evolution. They do not imply a presale, committed funding, or new economic conditions for the token.

Validation and operations

Autonomy claims must be supported by observable results and tests that include failures, recovery, and cost.

Dimension	Proposed acceptance evidence
Task quality	A deliverable checked against criteria defined before execution.
Recovery	Restart after several steps, reusing persisted results.
External effects	Loss of confirmation and a retry without duplicating the write.
Cost	Total consumption per task, including errors and retries.
Isolation	A specialist’s failure or saturation does not stop other work.

The evaluation set should represent product uses: files, terminal, browsing, memory, and delegation. It should preserve model and tool versions, the environment, and results. The same task should be tested with variations and common errors to identify when success depends on an unusually favorable condition.

Controls that support the work

The user needs to direct an execution and understand its effects. The design proposes consumption limits, credentials scoped to each capability, and recovery points. Reversible actions can proceed within the authorized scope; publishing, moving funds, or making changes that are difficult to undo requires the corresponding authorization.

Records should allow decisions and charges to be reconstructed without storing unnecessary secrets. Deduplication protects operations, and reconciliation protects the balance. These controls are integrated into the workflow so a task can continue, be corrected, or stop clearly.

Communicating results

An improvement should state what was measured, against which reference, and within which limits. This edition presents no new performance results and claims no certifications or audits. Commercial comparisons should accompany figures with a reproducible methodology and distinguish an experiment from a service guarantee.

Recovery and isolation criteria adapted from [1, pp. 26 to 30]. The availability of a published feature does not constitute an independent audit of its implementation.

References and scope

The proposed architecture builds on the reference paper; the descriptions of the product and Fuel use official Dulus sources.

[1] Temporal Durable orchestration for multi-agent AI systems. The 32-page PDF supplied as a reference. Capability boundaries, pp. 8 to 10 and 18 to 21; durable execution, pp. 11 to 17; coordination, pp. 21 to 26; validation, pp. 27 to 30.

[2] Dulus AI Product and ecosystem description. https://dulus.ai/

[3] Dulus Models Models, providers, and published holder-tier conditions. https://dulus.ai/models/

[4] Dulus Pricing Application access and included capabilities. https://dulus.ai/pricing/

[5] Fuel and DULUS Deposits, Fuel units, metering, settlement, and token identity. https://dulus.ai/fuel/

[6] Dulus Developers API, endpoints, and consumption receipts. https://dulus.ai/developers/

[7] Dulus open-source edition Public runtime repository. https://github.com/KevRojo/Dulus/

[8] Dulus About Project origins and Kevin Rojo as founder. https://dulus.ai/about/

Scope of this edition

Public sources consulted on October 19, 2026. Product features are described according to their public documentation; durable designs, evaluation criteria, and future phases are proposals in this edition. Rates and operational conditions should be checked against the current service version.

The economic scope covers $DULUS utility and Fuel consumption. It does not specify total or circulating supply, treasury allocations, historical distribution, vesting, mint authorities, or administrative rights. Those parameters require specific tokenomics documentation and onchain verification; they cannot be inferred from the Fuel crediting ratio.

Dulus AI · Edition 1.0 · English · October 19, 2026. This technical adaptation does not imply a commercial partnership with Temporal or that Temporal operates Dulus infrastructure.